



```
In [6]: import pandas as pd
import numpy as np
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

```
In [2]: #Data Preprocessing
```

```
In [7]: df=pd.DataFrame(pd.read_csv('House_Price_dataset - House_Price_dataset.csv'),se
df
```

```
Out[7]:
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	bas
0	13300000	7420	4.0	2.0	3.0	yes	no	
1	12250000	8960	4.0	4.0	4.0	yes	no	
2	12250000	9960	3.0	2.0	2.0	yes	no	
3	12215000	7500	4.0	2.0	2.0	yes	no	
4	11410000	7420	4.0	1.0	2.0	yes	yes	
...	...	...	...	...	...	...	...	...
540	1820000	3000	2.0	1.0	1.0	yes	no	
541	1767150	2400	3.0	1.0	1.0	no	no	
542	1750000	3620	2.0	1.0	1.0	yes	no	
543	1750000	2910	3.0	1.0	1.0	no	no	
544	1750000	3850	3.0	1.0	2.0	yes	no	

545 rows × 13 columns

```
In [8]: df.describe()
```

```
Out[8]:
```

	price	area	bedrooms	bathrooms	stories	parki
count	5.450000e+02	545.000000	540.000000	540.000000	543.000000	541.000000
mean	4.766729e+06	5150.541284	2.961111	1.285185	1.804788	0.691304
std	1.870440e+06	2170.141023	0.738779	0.502464	0.869011	0.860000
min	1.750000e+06	1650.000000	1.000000	1.000000	1.000000	0.000000
25%	3.430000e+06	3600.000000	2.000000	1.000000	1.000000	0.000000
50%	4.340000e+06	4600.000000	3.000000	1.000000	2.000000	0.000000
75%	5.740000e+06	6360.000000	3.000000	2.000000	2.000000	1.000000
max	1.330000e+07	16200.000000	6.000000	4.000000	4.000000	3.000000

```
In [9]: (df.isnull()).sum()
```

```
Out[9]: price          0
        area           0
        bedrooms       5
        bathrooms      5
        stories        2
        mainroad       0
        guestroom      0
        basement       0
        hotwaterheating 0
        airconditioning 0
        parking        4
        prefarea       0
        furnishingstatus 0
        dtype: int64
```

```
In [10]: df.select_dtypes(include=['int64', 'float64']).mean()
```

```
Out[10]: price          4.766729e+06
        area           5.150541e+03
        bedrooms       2.961111e+00
        bathrooms      1.285185e+00
        stories        1.804788e+00
        parking        6.913124e-01
        dtype: float64
```

```
In [11]: df.select_dtypes(include=['int64', 'float64']).median()
```

```
Out[11]: price          4340000.0
        area           4600.0
        bedrooms        3.0
        bathrooms       1.0
        stories         2.0
        parking         0.0
        dtype: float64
```

```
In [12]: df.select_dtypes(include=['int64', 'float64']).mean()
```

```
Out[12]: price          4.766729e+06
        area           5.150541e+03
        bedrooms       2.961111e+00
        bathrooms      1.285185e+00
        stories        1.804788e+00
        parking        6.913124e-01
        dtype: float64
```

```
In [13]: df.mode()
```

```
Out[13]:
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	baser
0	3500000	6000.0	3.0	1.0	2.0	yes	no	
1	4200000	NaN	NaN	NaN	NaN	NaN	NaN	

```
In [14]: df=df.fillna(df.mode().iloc[0])
         #df=df.fillna(df.select_dtypes(include=['int64', 'float64']).median())
```

```
In [15]: (df.isnull()).sum()
```

```
Out[15]: price          0
         area           0
         bedrooms       0
         bathrooms       0
         stories         0
         mainroad        0
         guestroom       0
         basement        0
         hotwaterheating  0
         airconditioning  0
         parking         0
         prefarea        0
         furnishingstatus 0
         dtype: int64
```

```
In [16]: x = df[['area']]
         y = df[['price']]
```

```
In [17]: x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.2,random_stat
```

```
In [18]: priceModel = LinearRegression().fit(x_train,y_train)
```

```
In [19]: priceModel.coef_
```

```
Out[19]: array([[464.29117589]])
```

```
In [20]: priceModel.intercept_
```

```
Out[20]: array([2405575.35717588])
```

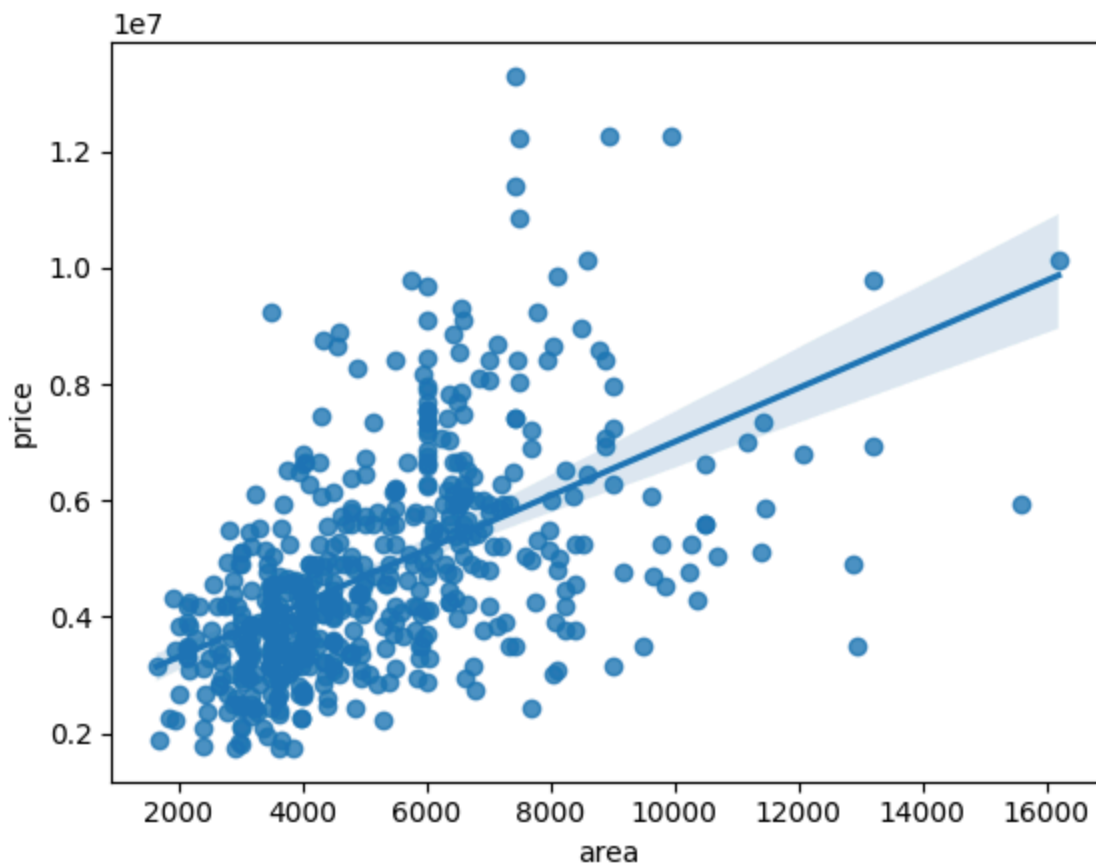
```
In [21]: y_tpred = priceModel.predict(x_train)
```

```
In [22]: train_mse = mean_squared_error(y_train,y_tpred)
         train_rmse = np.sqrt(train_mse)
         train_r2 = r2_score(y_train,y_tpred)
         print("Mean squared error",train_mse,"\nRMSE",train_rmse,"\nR^2 Score",train_r
```

```
Mean squared error 2487911673467.7773
RMSE 1577311.5334225441
R^2 Score 0.29859660823927303
```

```
In [23]: sns.regplot(x="area",y="price",data=df)
```

```
Out[23]: <Axes: xlabel='area', ylabel='price'>
```



```
In [24]: y_tpred = priceModel.predict(x_test)
```

```
In [26]: x2 = df.drop(columns=['price'])
x2 = x2.replace({'yes': 1, 'no': 0})
x2 = x2.replace({'furnished': 2, 'semi-furnished': 1, 'unfurnished': 0})
```

C:\Users\ARJUN\AppData\Local\Temp\ipykernel_20920\4006969702.py:2: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. To retain the old behavior, explicitly call `result.infer\_objects(copy=False)`. To opt-in to the future behavior, set `pd.set\_option('future.no\_silent\_downcasting', True)`

```
x2 = x2.replace({'yes': 1, 'no': 0})
```

C:\Users\ARJUN\AppData\Local\Temp\ipykernel_20920\4006969702.py:3: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. To retain the old behavior, explicitly call `result.infer\_objects(copy=False)`. To opt-in to the future behavior, set `pd.set\_option('future.no\_silent\_downcasting', True)`

```
x2 = x2.replace({'furnished': 2, 'semi-furnished': 1, 'unfurnished': 0})
```

```
In [27]: x2_train, x2_test, y_train, y_test = train_test_split(x2, y, test_size=0.2, random_s
```

```
In [28]: betterPriceModel = LinearRegression().fit(x2_train, y_train)
```

```
In [29]: y2t_pred = betterPriceModel.predict(x2_train)
```

```
In [30]: train2_mse = mean_squared_error(y_train,y2t_pred)
train2_rmse = np.sqrt(train2_mse)
train2_r2 = r2_score(y_train,y2t_pred)
print("Mean squared error",train2_mse,"\nRMSE",train2_rmse,"\nR^2 Score",train2_r2)
```

```
Mean squared error 1170811185288.5754
RMSE 1082040.2882002941
R^2 Score 0.6699195774389537
```

```
In [31]: x2_test = x2_test[x2_train.columns]

y2_pred=betterPriceModel.predict(x2_test)
```

```
In [32]: test2_mse = mean_squared_error(y_test,y2_pred)
test2_rmse = np.sqrt(test2_mse)
test2_r2 = r2_score(y_test,y2_pred)
print("Mean squared error",test2_mse,"\nRMSE",test2_rmse,"\nR^2 Score",test2_r2)
```

```
Mean squared error 989026605743.8591
RMSE 994498.1677931132
R^2 Score 0.691498523179515
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [33]: df.replace({
    'mainroad': {'yes': 1, 'no': 0},
    'guestroom': {'yes': 1, 'no': 0},
    'basement': {'yes': 1, 'no': 0},
    'hotwaterheating': {'yes': 1, 'no': 0},
    'airconditioning': {'yes': 1, 'no': 0},
    'prefarea': {'yes': 1, 'no': 0},
    'furnishingstatus': {'furnished': 2, 'semi-furnished': 1, 'unfurnished': 0}
}, inplace=True)
```

```
C:\Users\ARJUN\AppData\Local\Temp\ipykernel_20920\1405593542.py:1: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. To retain the old behavior, explicitly call `result.infer_objects(copy=False)`. To opt-in to the future behavior, set `pd.set_option('future.no_silent_downcasting', True)`
  df.replace({
```

```
In [34]: df
```

Out[34]:

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	bas
0	13300000	7420	4.0	2.0	3.0	1	0	
1	12250000	8960	4.0	4.0	4.0	1	0	
2	12250000	9960	3.0	2.0	2.0	1	0	
3	12215000	7500	4.0	2.0	2.0	1	0	
4	11410000	7420	4.0	1.0	2.0	1	1	
...	...	...	...	...	...	...	...	...
540	1820000	3000	2.0	1.0	1.0	1	0	
541	1767150	2400	3.0	1.0	1.0	0	0	
542	1750000	3620	2.0	1.0	1.0	1	0	
543	1750000	2910	3.0	1.0	1.0	0	0	
544	1750000	3850	3.0	1.0	2.0	1	0	

545 rows × 13 columns

In [35]: `df.mean()`

Out[35]:

price	4.766729e+06
area	5.150541e+03
bedrooms	2.961468e+00
bathrooms	1.282569e+00
stories	1.805505e+00
mainroad	8.587156e-01
guestroom	1.779817e-01
basement	3.504587e-01
hotwaterheating	4.587156e-02
airconditioning	3.155963e-01
parking	6.862385e-01
prefarea	2.348624e-01
furnishingstatus	9.302752e-01
dtype:	float64

In [36]: `df.median()`

```
Out[36]: price          4340000.0
         area           4600.0
         bedrooms       3.0
         bathrooms      1.0
         stories        2.0
         mainroad       1.0
         guestroom      0.0
         basement       0.0
         hotwaterheating 0.0
         airconditioning 0.0
         parking        0.0
         prefarea       0.0
         furnishingstatus 1.0
         dtype: float64
```

```
In [37]: df.mode()
```

```
Out[37]:
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	baser
0	3500000	6000.0	3.0	1.0	2.0	1.0	0.0	
1	4200000	NaN	NaN	NaN	NaN	NaN	NaN	

```
In [38]: (df.isnull()).sum()
```

```
Out[38]: price          0
         area          0
         bedrooms      0
         bathrooms     0
         stories       0
         mainroad      0
         guestroom     0
         basement      0
         hotwaterheating 0
         airconditioning 0
         parking       0
         prefarea      0
         furnishingstatus 0
         dtype: int64
```

```
In [39]: df=df.fillna(df.mode().iloc[0])
```

```
In [40]: (df.isnull()).sum()
```

```
Out[40]: price          0
         area          0
         bedrooms      0
         bathrooms     0
         stories       0
         mainroad      0
         guestroom     0
         basement      0
         hotwaterheating 0
         airconditioning 0
         parking       0
         prefarea      0
         furnishingstatus 0
         dtype: int64
```

```
In [41]: x = df[['area']]
         y = df[['price']]
```

```
In [42]: x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.2,random_stat
```

```
In [43]: priceModel = LinearRegression().fit(x_train,y_train)
```

```
In [44]: priceModel.coef_
```

```
Out[44]: array([[420.83286393]])
```

```
In [46]: priceModel.intercept_
```

```
Out[46]: array([2530732.96254749])
```

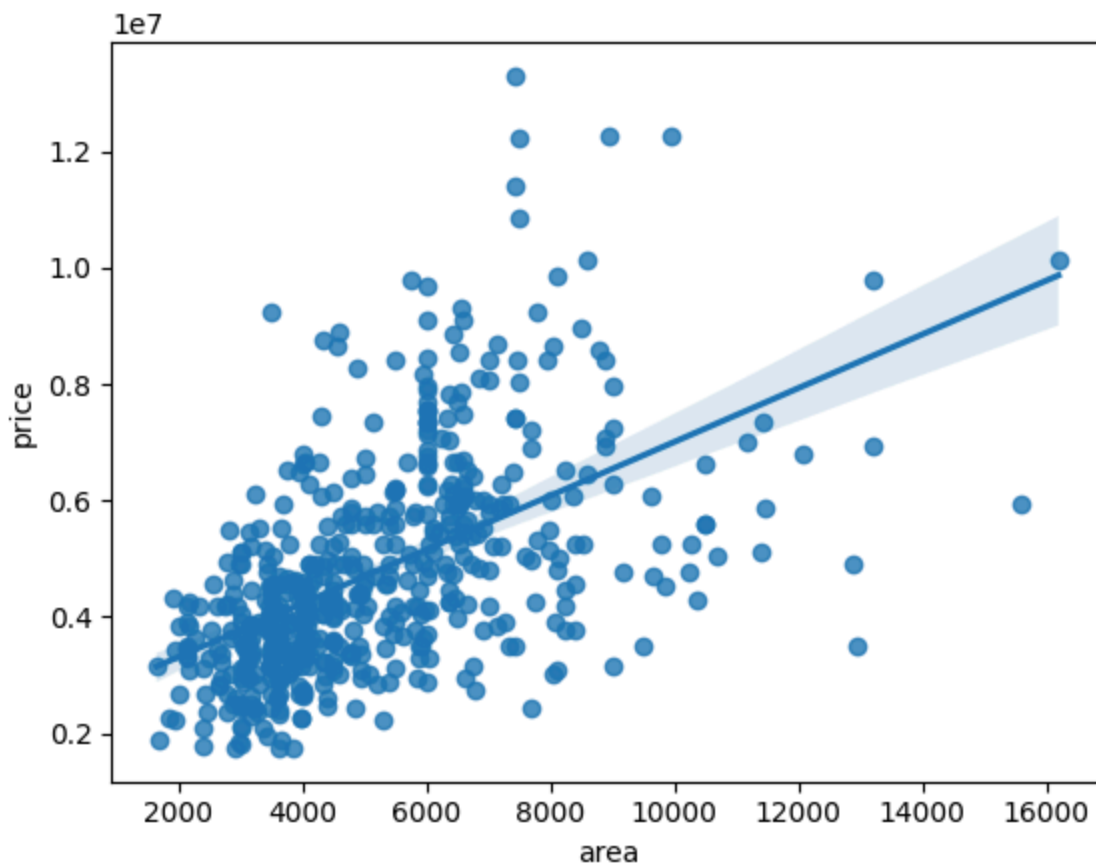
```
In [47]: y_tpred = priceModel.predict(x_train)
```

```
In [48]: train_mse = mean_squared_error(y_train,y_tpred)
         train_rmse = np.sqrt(train_mse)
         train_r2 = r2_score(y_train,y_tpred)
         print("Mean squared error",train_mse,"\nRMSE",train_rmse,"\nR^2 Score",train_r
```

```
Mean squared error 2231722397483.0713
RMSE 1493895.0423249523
R^2 Score 0.27316451038299494
```

```
In [49]: sns.regplot(x="area",y="price",data=df)
```

```
Out[49]: <Axes: xlabel='area', ylabel='price'>
```

```
In [50]: y_tpred = priceModel.predict(x_test)
```

```
In [51]: x2 = df.drop(columns=['price'])
x2_train,x2_test,y_train,y_test = train_test_split(x2,y,test_size=0.2,random_s
```

```
In [52]: betterPriceModel = LinearRegression().fit(x2_train,y_train)
```

```
In [53]: y2t_pred=betterPriceModel.predict(x2_train)
train2_mse = mean_squared_error(y_train,y2t_pred)
train2_rmse = np.sqrt(train2_mse)
train2_r2 = r2_score(y_train,y2t_pred)
print("Mean squared error",train2_mse,"\nRMSE",train2_rmse,"\nR^2 Score",train
```

Mean squared error 929876968560.7394

RMSE 964301.2851597469

R^2 Score 0.6971542775706949

```
In [54]: x2_test = x2_test[x2_train.columns]

y2_pred=betterPriceModel.predict(x2_test)
```

```
In [55]: test2_mse = mean_squared_error(y_test,y2_pred)
test2_rmse = np.sqrt(test2_mse)
test2_r2 = r2_score(y_test,y2_pred)
print("Mean squared error",test2_mse,"\nRMSE",test2_rmse,"\nR^2 Score",test2_r
```

Mean squared error 2032257490178.0037
RMSE 1425572.6884932958
R^2 Score 0.595224504692778

```
In [57]: inarea = int(input('Enter area '))
inbedrooms = float(input('Enter number of bedrooms '))
inbathrooms = float(input('Enter number of bathrooms'))
instories = float(input('Enter number of stories(floors) '))
inmainroad = int(input('Is house on main road (yes:1/no:0)'))
inguestroom = int(input('Does house have guest room (yes:1/no:0)'))
inbasement = int(input('Does house have a basement (yes:1/no:0)'))
inhotwaterheating = int(input('Does house have hot water heating (yes:1/no:0)'))
inairconditioning = int(input('Does house have AC(yes:1/no:0)'))
inparking = float(input("How many cars can parking accomodate (enter 0 if none)"))
inprefarea = int(input("Is house in preferred area (yes:1/no:0) "))
infurnishingstatus = int(input("Furnishing status of house (furnished:2,semi-furnished:1,unfurnished:0)"))
print("The price of your house depending on all factors is",betterPriceModel.predict([inarea,inbedrooms,inbathrooms,instories,inmainroad,inguestroom,inbasement,inhotwaterheating,inairconditioning,inparking,inprefarea,infurnishingstatus]))
```

The price of your house depending on all factors is 7711414.2446102295

C:\Users\ARJUN\AppData\Roaming\Python\Python311\site-packages\sklearn\utils\validation.py:2749: UserWarning: X does not have valid feature names, but LinearRegression was fitted with feature names
warnings.warn(

In []: